

BANGALORE CITY UNIVERSITY

CA-C5P: DATA STRUCTURES LAB

LAB PROGRAM: 1

/*Given {4,7,3,2,1,7,9,0} find the location of 7 using Linear and Binary search and also display its first occurrence */

```
#include<stdio.h>
#include<conio.h>
int binarySearch(int arr[],int size, int target)
{
    int left=0;
    int right=size-1;
    while(left<=right)
    {
        int mid=left+(right-left)/2;
        if(arr[mid]==target)
        {
            return mid;
        }
        else if(arr[mid]<target)
        {
            left=mid+1;
        }
        else
        {
            right=mid-1;
        }
    }
    return -1; //Element Not Found
}
//Linear Search Function
int linearSearch(int i,int arr[],int size,int target)
{
    for(i=0;i<size;i++)
    {
        if(arr[i]==target)
        {
            return i;
        }
    }
    return -1; //Element not found
}
int main()
```

```

{
int arr[]={4,7,3,2,1,7,9};
int size=sizeof(arr)/sizeof(arr[0]);
int i,j,temp,location,FO,target=7;
int binaryLocation;
int linearLocation;
clrscr();
//Sort the array
for(i=0;i<size-1;i++)
{
for(j=0;j<size-i-1;j++)
{
if(arr[j]>arr[j+1])
{
//Swap arr[j] and arr[j+1]
temp=arr[j];
arr[j]=arr[j+1];
arr[j+1]=temp;
}
}
}
printf("Sorted array:\n");
for(i=0;i<size;i++)
printf("%d\t",arr[i]);
binaryLocation=binarySearch(arr,size,target);
linearLocation=linearSearch(i,arr,size,target);
if(binaryLocation!=-1)
{
printf("\nUsing binary search, the element %d is found at index %d\n",target,binaryLocation);
FO=binaryLocation;
while(FO>0&&arr[FO-1]==target)
{
FO--;
}
printf("The First Occurrence of %d is at index %d\n",target,FO);
}
else
{
printf("Element %d not found using binary search\n",target);
}
if(linearLocation!=-1)
{
printf("Using linear search, the element %d is found at index:%d\n",target,linearLocation);
}
}

```

```

else
{
printf("Element %d is not found using linear search\n",target);
}
getch();
return 0;
}

```

OUTPUT:

```

Sorted array:
1      2      3      4      7      7      9
Using binary search, the element 7 is found at index 5
The First Occurrence of 7 is at index 4
Using linear search, the element 7 is found at index:4

```

LAB PROGRAM: 2

//Given {5,3,1,6,0,2,4} order the numbers in ascending order using Bubble Sort Algorithm.

```

#include<stdio.h>
#include<conio.h>
int main()
{
int arr[]={5,3,1,6,0,2,4};
int n=7,i,j,temp,swapped;
clrscr();
printf("Original Array\n");
for(i=0;i<n;i++)
{
printf("%d\t",arr[i]);
}
printf("\n\n");
for(i=0;i<n-1;i++)
{
swapped=0;
for(j=0;j<n-1-i;j++)
{
if(arr[j]>arr[j+1])
{
//Swap arr[j] and arr[j+1]
temp=arr[j];

```

```

arr[j]=arr[j+1];
arr[j+1]=temp;
swapped=1;
}
}
printf("Pass %d\t:",i+1);
for(j=0;j<n;j++)
{
printf("\t%d",arr[j]);
}
printf("\n");
// If no two elements were swapped in the inner loop, the array is already sorted.
if(swapped==0)
{
break;
}
}
printf("\nSorted Array : ");
for(i=0;i<n;i++)
{
printf("\t%d",arr[i]);
}
getch();
return 0;
}

```

OUTPUT :

```

DOSBox 0.74, Cpu speed: max 100% cycles, Frameskip 0, Progr...
Original Array
5      3      1      6      0      2      4
Pass 1 :      3      1      5      0      2      4      6
Pass 2 :      1      3      0      2      4      5      6
Pass 3 :      1      0      2      3      4      5      6
Pass 4 :      0      1      2      3      4      5      6
Pass 5 :      0      1      2      3      4      5      6
Sorted Array : 0      1      2      3      4      5      6_

```

LAB PROGRAM: 3

/* Perform the Insertion and Selection Sort on the input {75,8,1,16,48,3,7,0} and display the output in descending order.*/

```

#include<stdio.h>
#include<conio.h>

```

```

void selectionSort(int arr[],int n,int i,int j)
{
for(i=0;i<n-1;i++)
{
int maxIndex=i;
for(j=i+1; j<n;j++)
{
if(arr[j]>arr[maxIndex])
{
maxIndex=j;
}
}
if(maxIndex !=i)
{
int temp=arr[i];
arr[i]=arr[maxIndex];
arr[maxIndex]=temp;
}
}
}
//Function to perform Insertion Sort
void insertionSort(int arr[],int n,int i,int j)
{
for(i=1;i<n;i++)
{
int key=arr[i];
j=i-1;
while(j>=0 && arr[j]<key)
{
arr[j+1]=arr[j];
j--;
}
arr[j+1]=key;
}
}
int main()
{
int arr[]={75,8,1,16,48,3,7,0};
int n=sizeof(arr)/sizeof(arr[0]);
int i,j;
clrscr();
printf("Original Array:\n");
for(i=0;i<n;i++)
{

```

```

printf("\t%d",arr[i]);
}
selectionSort(arr,n,i,j);
printf("\n\nDescending Order (Selection Sort):\n");
for(i=0;i<n;i++)
{
printf("\t%d",arr[i]);
}
//Insertion Sort
insertionSort(arr,n,i,j);
printf("\nDescending Order (Insertion Sort):\n");
for(i=0;i<n;i++)
{
printf("%d\t",arr[i]);
}
getch();
return 0;
}

```

OUTPUT :

```

DOSBox 0.74, Cpu speed: max 100% cycles, Frameskip 0, Progr...
Original Array:
75 8 1 16 48 3 7 0
Descending Order (Selection Sort):
75 48 16 8 7 3 1 0
Descending Order (Insertion Sort):
75 48 16 8 7 3 1 0_

```

LAB PROGRAM: 4

/* Write a program to insert the elements {61,16,8,27} into singly linked list and delete 8,61,27 from the list. Display your list after each insertion and deletion*/

```

#include<stdio.h>
#include<conio.h>
struct node
{
int data;
struct node*next;
};

void displaylist(struct node*head)
{
struct node*current=head;

```

```

while(current!=NULL)
{
printf("%d->",current->data);
current=current->next;
}
printf("Null\n");
}
struct node*insertelement(struct node*head,int data)
{
struct node*newnode=(struct node*)malloc(sizeof(struct node));
newnode->data=data;
newnode->next=head;
return newnode;
}
struct node*deleteelement(struct node*head,int data)
{
struct node*current=head;
struct node*prev=NULL;
while(current!=NULL)
{
if(current->data==data)
{
if(prev==NULL)
{
head=current->next;
}
else
{
prev->next=current->next;
}
free(current);
return head;
}
prev=current;
current=current->next;
}
return head;
}
int main()
{
struct node*head=NULL;
clrscr();
head=insertelement(head,61);
printf("Linked list after insertion 61:");
displaylist(head);
head=insertelement(head,16);

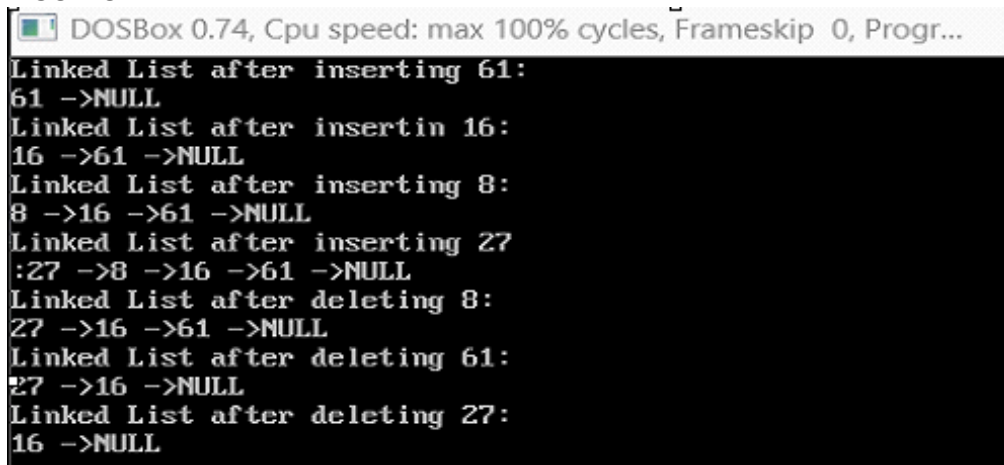
```

```

printf("Linked list after insertion 16:");
displaylist(head);
head=insertelement(head,8);
printf("Linked list after insertion 8");
displaylist(head);
head=deleteelement(head,8);
printf("Linked list after deletion 8:");
displaylist(head);
head=deleteelement(head,61);
printf("Linked list after deletion 61:");
displaylist(head);
head=deleteelement(head,16);
printf("Linked list after deletion 16:");
displaylist(head);
while(head!=NULL)
{
    struct node*temp=head;
    head=head->next;
    free(temp);
}
getch();
return 0;
}

```

OUTPUT :



```

DOSBox 0.74, Cpu speed: max 100% cycles, Frameskip 0, Progr...
Linked List after inserting 61:
61 ->NULL
Linked List after insertin 16:
16 ->61 ->NULL
Linked List after inserting 8:
8 ->16 ->61 ->NULL
Linked List after inserting 27
:27 ->8 ->16 ->61 ->NULL
Linked List after deleting 8:
27 ->16 ->61 ->NULL
Linked List after deleting 61:
27 ->16 ->NULL
Linked List after deleting 27:
16 ->NULL

```

LAB PROGRAM: 5

/*Write a program to insert the elements {61,16,8,27} into linear queue and delete three elements from the list.Display your list after each insertion and deletion*/

```

#include<stdio.h>
#include<conio.h>
#define maxsize 5

```



```

struct queue
{
int front,rear;
int element[maxsize];
};
void initqueue(struct queue*queue)
{
queue->front=-1;
queue->rear=-1;
}
int isfull(struct queue*queue)
{
return(queue->rear==maxsize-1);
}
int isempty(struct queue*queue)
{
return(queue->front== -1);
}
void enqueue(struct queue*queue,int item)
{
if(!isfull(queue))
{
if(isempty(queue))
{
queue->front=0;
}
queue->rear++;
queue->element[queue->rear]=item;
}
else
{
printf("Queue is full cannot enqueue %d\n",item);
}
}
int dequeue(struct queue*queue)
{
int item=-1;
if(!isempty(queue))
{

```

```

item=queue->element[queue->front];
if(queue->front==queue->rear)
{
queue->front=-1;
queue->rear=-1;
}
else
{
queue->front++;
}
}
return item;
}
void display(struct queue*queue)
{
int i;
if(!isempty(queue))
{
printf("Queue:\t");
for(i=queue->front;i<=queue->rear;i++)
{
printf("%d\t",queue->element[i]);
}
printf("\n");
}
else
{
printf("Queue is empty\n");
}
}
void main()
{
int deletedelement;
struct queue queue;
initqueue(&queue);
clrscr();
enqueue(&queue,61);
printf("Queue after inserting 61:\n");
display(&queue);

```

```

enqueue(&queue,16);
printf("Queue after inserting 16:\n");
display(&queue);
enqueue(&queue,8);
printf("Queue after inserting 8:\n");
display(&queue);
enqueue(&queue,27);
printf("Queue after inserting 27:\n");
display(&queue);

deletedelement=dequeue(&queue);
printf("Dequeue: %d\n",deletedelement);
display(&queue);
deletedelement=dequeue(&queue);
printf("Dequeue: %d\n",deletedelement);
display(&queue);
deletedelement=dequeue(&queue);
printf("Dequeue: %d\n",deletedelement);
display(&queue);
getch();
}

```

OUTPUT :

```

Queue after inserting 61:
Queue: 61
Queue after inserting 16:
Queue: 61 16
Queue after inserting 8:
Queue: 61 16 8
Queue after inserting 27:
Queue: 61 16 8 27
Dequeue: 61
Queue: 16 8 27
Dequeue: 16
Queue: 8 27
Dequeue: 8
Queue: 27

```

LAB PROGRAM: 6

/*Write a c program to insert the elements {61,16,8,27} into circular queue and delete 4 elements from the list. Display your list after each insertion and deletion*/

```

#include<stdio.h>
#include<conio.h>

```

```

#define MAX_SIZE 4
//Circular Queue structure
struct cirque
{
    int items[MAX_SIZE];
    int front,rear;
};
//Function to initialize the circular queue
void initque(struct cirque *que)
{
    que->front=-1;
    que->rear=-1;
}
//Function to check if the queue is empty
int isEmpty(struct cirque *que)
{
    return(que->front==-1 && que->rear==-1);
}
int isFull(struct cirque *que)
{
    return(que->front==(que->rear+1)%MAX_SIZE);
}
//Function to insert an element into the circular queue
void enqueue(struct cirque *que,int value)
{
    if(isFull(que))
    {
        printf("Queue is full. Cannot insert %d.\n",value);
        return;
    }
    if(isEmpty(que))
    {
        que->front=0;
        que->rear=0;
    }
    else
    {
        que->rear=(que->rear+1)%MAX_SIZE;
    }
    que->items[que->rear]=value;
    printf("Inserted %d into the the queue.\n",value);
}
//Function to delete an element from the circular queue
void deque(struct cirque *que)
{
    if(isEmpty(que))
    {
        printf("Queue is empty. Cannot delete.\n");
        return;
    }
    printf("Deleted %d from the queue.\n",que->items[que->front]);
}

```

```

if(que->front==que->rear)
{
//Last element in the queue
initque(que);
}
else
{
que->front=(que->front+1)%MAX_SIZE;
}
}
//Function to display the elements in the circular queue
void display(struct cirque *que)
{
int i;
if(isEmpty(que))
{
printf("Queue is empty\n");
return;
}
printf("Current state of the queue:");
i=que->front;
do
{
printf(" %d\t",que->items[i]);
i=(i+1)%MAX_SIZE;
}while(i!=(que->rear+1)%MAX_SIZE);
printf("\n");
}
//Main function
int main()
{
struct cirque que;
clrscr();
initque(&que);
//Inserting elements into the circular queue
enqueue(&que,61);
display(&que);
enqueue(&que,16);
display(&que);
enqueue(&que,8);
display(&que);
enqueue(&que,27);
display(&que);
//Deleting elements from the circular que
dequeue(&que);
display(&que);
dequeue(&que);
display(&que);
dequeue(&que);
display(&que);
dequeue(&que);

```

```

display(&que);
getch();
return 0;
}

```

OUTPUT:

```

Inserted 61 into the the queue.
Current state of the queue:61
Inserted 16 into the the queue.
Current state of the queue:61 16
Inserted 8 into the the queue.
Current state of the queue:61 16 8
Inserted 27 into the the queue.
Current state of the queue:61 16 8 27
Deleted 61 from the queue.:
Current state of the queue:16 8 27
Deleted 16 from the queue.:
Current state of the queue:8 27
Deleted 8 from the queue.:
Current state of the queue:27
Deleted 27 from the queue.:
Queue is empty.

```

LAB PROGRAM: 7

/* Write a c program to insert the elements {61,16,8,27} into ordered singly linked list and delete 8,61,27 from the list. Display your list after each insertion and deletion.*/

```

#include <stdio.h>
#include <stdlib.h>
#include <conio.h>
// Node structure for the linked list
struct Node
{
int data;
struct Node* next;
};
// Function to create a new node
struct Node*temp;
struct Node*prev;
struct Node* createNode(int value)
{
struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
newNode->data = value;
newNode->next = NULL;
return newNode;
}
// Function to insert an element into the ordered linked list
void insertOrdered(struct Node** head, int value)

```

```

{
struct Node* newNode = createNode(value);
if (*head == NULL || value < (*head)->data)
{
newNode->next = *head; *head = newNode;
}
else
{
struct Node* current = *head;
while (current->next != NULL && current->next->data < value)
{
current = current->next;
}
newNode->next = current->next;
current->next = newNode;
}
printf("Inserted %d into the list.\n", value);
}
// Function to delete a specific element from the linked list
void deleteNode(struct Node** head, int value)
{
if (*head == NULL)
{
printf("List is empty. Cannot delete %d.\n", value);
return;
}
temp = *head;
prev = NULL;
// Find the node to be deleted
while (temp != NULL && temp->data != value)
{
prev = temp;
temp = temp->next;
}
if (temp == NULL)
{
printf("%d not found in the list. Cannot delete.\n", value);
return;
}
if (prev == NULL)
{
// Deleting the head node
*head = temp->next;
}
else
{
// Deleting a non-head node
prev->next = temp->next;
}
free(temp);
printf("Deleted %d from the list.\n", value);
}

```

```

}
// Function to display the elements in the linked list
void displayList(struct Node* head)
{
if (head == NULL)
{
printf("List is empty.\n");
return;
}
printf("Current state of the list: ");
while (head != NULL)
{
printf("%d ", head->data);
head = head->next;
}
printf("\n");
}
// Main function
int main()
{
struct Node* head = NULL;
clrscr();
// Inserting elements into the ordered linked list
insertOrdered(&head, 61);
displayList(head);
insertOrdered(&head, 16);
displayList(head);
insertOrdered(&head, 8);
displayList(head);
insertOrdered(&head, 27);
displayList(head);
// Deleting elements from the linked list
deleteNode(&head, 8);
displayList(head);
deleteNode(&head, 61);
displayList(head);
deleteNode(&head, 27);
displayList(head);
getch();
return 0;
}

```

OUTPUT:


```

Inserted 61 into the list.
Current state of the list: 61
Inserted 16 into the list.
Current state of the list: 16 61
Inserted 8 into the list.
Current state of the list: 8 16 61
Inserted 27 into the list.
Current state of the list: 8 16 27 61
Deleted 8 from the list.
Current state of the list: 16 27 61
Deleted 61 from the list.
Current state of the list: 16 27
Deleted 27 from the list.
Current state of the list: 16

```

LAB PROGRAM: 8

/*Write a program to add $6x^3+10x^2+0x+5$ and $4x^2+2x+14x^2+2x+1$ */

```

#include<stdio.h>
#include<stdlib.h>
#include<conio.h>
struct node
{
int coefficient;
int exponent;
struct node*next;
};
struct node*poly1;
struct node*poly2;
struct node*result;
struct node*createnode(int coefficient,int exponent)
{
struct node*newnode=(struct node*)malloc(sizeof(struct node));
newnode->coefficient=coefficient;
newnode->exponent=exponent;
newnode->next=NULL;
return newnode;
}
struct node*addpolynomials(struct node*poly1,struct node*poly2)
{
struct node*result=NULL;
struct node*temp=NULL;
while(poly1!=NULL||poly2!=NULL)
{
int coeff1=(poly1!=NULL)?poly1->coefficient:0;
int coeff2=(poly2!=NULL)?poly2->coefficient:0;
int expo1=(poly1!=NULL)?poly1->exponent:-1;
int expo2=(poly2!=NULL)?poly2->exponent:-2;
if(expo1==expo2)
{

```

```

int sumcoefficient=coeff1+coeff2;
if(sumcoefficient!=0)
{
temp=createnode(sumcoefficient,expo1);
temp->next=result;
result=temp;
}
poly1=poly1->next;
poly2=poly2->next;
}
else if(expo1>expo2)
{
temp=createnode(coeff1,expo1);
temp->next=result;
result=temp;
poly1=poly1->next;
}
else
{
temp=createnode(coeff2,expo2);
temp->next=result;
result=temp;
poly2=poly2->next;
}
}
return result;
}
void displaypolynomial(struct node*poly)
{
while(poly!=NULL)
{
printf("%d x^%d",poly->coefficient,poly->exponent);
poly=poly->next;
if(poly!=NULL)
{
printf("+");
}
}
printf("\n");
}
void freepolynomial(struct node*poly)
{
struct node*temp;
while(poly!=NULL)
{

```

```

temp=poly;
poly=poly->next;
free(temp);
}
}
int main()
{
poly1=createnode(6,3);
clrscr();
poly1->next=createnode(10,2);
poly1->next->next=createnode(0,1);
poly1->next->next->next=createnode(5,0);
poly2=createnode(4,2);
poly2->next=createnode(2,1);
poly2->next->next=createnode(1,0);
printf("Polynomial 1:");
displaypolynomial(poly1);
printf("Polynomial 2:");
displaypolynomial(poly2);
result=addpolynomials(poly1,poly2);
printf("Result:");
displaypolynomial(result);
freepolynomial(result);
freepolynomial(poly1);
freepolynomial(poly2);
freepolynomial(result);
getch();
return 0;
}

```

OUTPUT:

```

Polynomial 1: 6x^3 + 10x^2 + 0x^1 + 5x^0
Polynomial 2: 4x^2 + 2x^1 + 1x^0
Result: 6x^3 + 2x^2 + 14x^1 + 6x^0

```

LAB PROGRAM: 9

/* Write a program to push 5,9,34,17,32 into stack and pop 3 times from the stack, also display the popped numbers */

```

#include<stdio.h>
#include<conio.h>
#define maxsize 10
int stack[maxsize];

```

```

int top=-1;
int isfull()
{
return top==maxsize-1;
}
int isempty()
{
return top==-1;
}
void push(int item)
{
if(isfull())
{
printf("Stack is full cannot push %d\n",item);
}
else
{
top++;
stack[top]=item;
printf("Push %d onto to stack\n",item);
}
}
int pop()
{
if(isempty())
{
printf("Stack is empty cannot pop()\n:");
return-1;
}
else
{
int item=stack[top];
top--;
return item;
}
}
void main()
{
int i,poppedele;
clrscr();
push(5);
push(12);
push(22);
push(34);
push(55);
for(i=0;i<3;i++)
{
poppedele=pop();
}
}

```

```

if(poppedele!=-1)
{
printf("Popped %d from the stack\n",poppedele);
}
}
getch();
}

```

OUTPUT:

```

Pushed 5 onto to the stack.
Pushed 9 onto to the stack.
Pushed 34 onto to the stack.
Pushed 17 onto to the stack.
Pushed 32 onto to the stack.
Popped 32 from the stack.
Popped 17 from the stack.
Popped 34 from the stack.

```

LAB PROGRAM: 10

/*Write a recursive program to find GCD of 4,6,8*/

```

#include<stdio.h>
#include<conio.h>
int findgcd(int a,int b)
{
if(b==0)
{
return a;
}
else
{
return findgcd(b,a%b);
}
}
void main()
{
int n1=4,n2=6,n3=8;
int gcd1,gcd2;
clrscr();
gcd1=findgcd(n1,n2);
gcd2=findgcd(gcd1,n3);
printf("GCD of %d %d %d is:%d\n",n1,n2,n3,gcd2);
getch();
}

```

OUTPUT:

GCD of 4 6 and 8 is:2

LAB PROGRAM: 11

/*Write a program to insert the elements {5,7,0,6,3,9} into circular queue and delete 6,9&5 from it(using linked list)*/

```
#include<stdio.h>
#include<stdlib.h>
#include<conio.h>
struct node
{
int data;
struct node*next;
};
struct circularqueue
{
struct node*front;
struct node*rear;
};
struct node*current;
struct node*prev;
struct node*current;
struct node*createnode(int value)
{
struct node*newnode=(struct node*)malloc(sizeof(struct node));
newnode->data=value;
newnode->next=NULL;
return newnode;
}
void initializequeue(struct circularqueue*queue)
{
queue->front=NULL;
queue->rear=NULL;
}
int isempty(struct circularqueue*queue)
{
return(queue->front==NULL&&queue->rear==NULL);
}
void enqueue(struct circularqueue*queue,int value)
{
struct node*newnode=createnode(value);
if(isempty(queue))
{
```

```

queue->front=newnode;
queue->rear=newnode;
newnode->next=newnode;
}
else
{
newnode->next=queue-> front;
queue->rear->next= newnode;
queue->rear=newnode;
}
printf("Inserted %d into the queue:\n",value);
}
void dequeue(struct circularqueue*queue,int value)
{
if(isempty(queue))
{
printf("Queue is empty cannot delete %d.\n",value);
return;
}
current=queue->front;
prev=NULL;
do
{
if(current->data==value)
{
break;
}
prev=current;
current=current->next;
}
while(current!=queue->front);
if(current==queue->front)
{
if(queue->front->next==queue->front)
{
initializequeue(queue);
}
else
{
queue->front=queue->front->next;
queue->rear->next=queue->front;
}
}
else
{

```

```

prev->next=current->next;
if(current==queue->rear)
{
queue->rear=prev;
}
}
free(current);
printf("Delete %d from the queue.\n",value);
}
void displayqueue(struct circularqueue*queue)
{
if(isempty(queue))
{
printf("Queue is empty.\n");
return;
}
printf("Current state of the queue:");
current=queue->front;
do
{
printf("%d\t",current->data);
current=current->next;
}
while(current!=queue->front);
printf("\n");
}
int main()
{
struct circularqueue queue;
initializequeue(&queue);
clrscr();
enqueue(&queue,5);
displayqueue(&queue);
enqueue(&queue,7);
displayqueue(&queue);
enqueue(&queue,0);
displayqueue(&queue);
enqueue(&queue,6);
displayqueue(&queue);
enqueue(&queue,3);
displayqueue(&queue);
enqueue(&queue,9);
displayqueue(&queue);

dequeue(&queue,6);

```



```

displayqueue(&queue);
dequeue(&queue,9);
displayqueue(&queue);
dequeue(&queue,5);
displayqueue(&queue);
getch();
return 0;
}

```

OUTPUT:

```

Inserted 5 into the queue.
Current state of the queue: 5
Inserted 7 into the queue.
Current state of the queue: 5 7
Inserted 0 into the queue.
Current state of the queue: 5 7 0
Inserted 6 into the queue.
Current state of the queue: 5 7 0 6
Inserted 3 into the queue.
Current state of the queue: 5 7 0 6 3
Inserted 9 into the queue.
Current state of the queue: 5 7 0 6 3 9
Deleted 6 from the queue.
Current state of the queue: 5 7 0 3 9
Deleted 9 from the queue.
Current state of the queue: 5 7 0 3
Deleted 5 from the queue.
Current state of the queue: 7 0 3

```

LAB PROGRAM: 12

/*Write a program to convert an infix expression $x^y/(5*z)+2$ to its postfix expression*/

```

#include<stdio.h>
#include<conio.h>
#include<string.h>
#define MAX_SIZE 50
int mystack[MAX_SIZE];
int top;
//Function to check if a character is an operator
int isOperator(char c)
{
return(c=='+' || c=='-' || c=='*' || c=='/' || c=='^');
}
//Function to get the precedence of an operator
int getPrecedence(char operator)
{

```

```

if(operator == '^')
return 3;
if(operator == '*' || operator == '/')
return 2;
if(operator == '+' || operator == '-')
return 1;
return 0;
}
//Function to push an element onto the stack
void push(int item)
{
if(top == MAX_SIZE-1)
{
printf("Stack Overflow\n");
}
else
{
mystack[++top]=item;
}
}
//Function to pop an element from the stack
int pop()
{
if(top == -1)
{
printf("Stack Underflow\n");
return -1;
}
else
{
return mystack[top--];
}
}
void infixPostfix(char infix[])
{
char postfix[MAX_SIZE],i;
int postfixIndex=0;
for(i=0;infix[i]!='\0'; i++)
{
char c = infix[i];
if(c=='(')
{
push(c);

```

```

}
else if(c == ')')
{
while(top>=0 && mystack[top]!='(')
{
postfix[postfixIndex++]=pop();
}
pop(); //Pop the '('
}
else if(!isOperator(c))
{
postfix[postfixIndex++]=c;
}
else
{
while(top>=0 && getPrecedence(mystack[top])>=getPrecedence(c))
{
postfix[postfixIndex++]=pop();
}
push(c);
}
}
while(top>=0)
{
postfix[postfixIndex++]=pop();
}
postfix[postfixIndex++] = '\0';
printf("Postfix Expression: %s\n",postfix);
}
int main()
{
char infix[MAX_SIZE];
clrscr();
printf("Enter the infix expression (without spaces):");
gets(infix);
infixPostfix(infix);
getch();
return 0;
}
OUTPUT:

```

```
Enter the infix expression (without spaces): x^y(5*z)+2
Postfix Expression: xy5z*^2+
```

LAB PROGRAM: 13

/* Write a program to evaluate a post-fix expression 5 3 +8 2 - *./

```
#include<stdio.h>
#include<conio.h>
#define maxsize 50
int stack[maxsize];
int top=-1;
int isoperand(char c)
{
    return(c>='0'&&c<'9');
}
void push(int item)
{
    if(top==maxsize-1)
    {
        printf("Stack overflow\n");
        exit(1);
    }
    stack[++top]=item;
}
int pop()
{
    if(top==--1)
    {
        printf("Stack underflow\n");
        exit(1);
    }
    return stack[top--];
}
int evaluatepostfix(char postfix[])
{
    int i=0;
    while(postfix[i]!='\0')
    {
        char c=postfix[i];
        if(isoperand(c))
        {
```

```

push(c-'0');
}
else
{
int operand2=pop();
int operand1=pop();
int result;
switch(c)
{
case '+':result=operand1+operand2;
break;
case '-':result=operand1-operand2;
break;
case '*':result=operand1*operand2;
break;
case '/':result=operand1/operand2;
break;
default:
printf("Invalid operand:%c\n",c);
exit(1);
}
push(result);
}
i++;
}
return pop();
}
void main()
{
char postfix[]="53+82-*" ;
int result;
clrscr();
result=evaluatepostfix(postfix);
printf("Result of the postfix of the postfix expression:%d\n",result);
getch();
}

```

OUTPUT:

RESULT OF THE POSTFIX OF THE POSTFIX EXPRESSION:48

/*Write a program to create a binary tree with the element {18,15,40,50,30,17,41} after creation insert 45 and 19 into tree and delete 15,17 and 41 from tree.Display the treeon each insertion and deletion operation*/

```
#include <stdio.h>
#include <stdlib.h>
#include <conio.h>
struct Node
{
    int data;
    struct Node *left;
    struct Node *right;
};
struct Node *temp;
struct Node *createNode(int value)
{
    struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));
    newNode->data = value;
    newNode->left = newNode->right = NULL;
    return newNode;
}

struct Node *insert(struct Node *root, int value)
{
    if (root == NULL)
        return createNode(value);

    if (value < root->data)
        root->left = insert(root->left, value);
    else if (value > root->data)
        root->right = insert(root->right, value);

    return root;
}

struct Node *minValueNode(struct Node *node)
{
    struct Node *current = node;
    while (current->left != NULL)
        current = current->left;
    return current;
}
```

```

struct Node *deleteNode(struct Node *root, int value)
{
    if (root == NULL)
        return root;

    if (value < root->data)
        root->left = deleteNode(root->left, value);
    else if (value > root->data)
        root->right = deleteNode(root->right, value);
    else
    {
        if (root->left == NULL)
        {
            struct Node *temp = root->right;
            free(root);
            return temp;
        }
        else if (root->right == NULL)
        {
            struct Node *temp = root->left;
            free(root);
            return temp;
        }
        temp = minValueNode(root->right);
        root->data = temp->data;
        root->right = deleteNode(root->right, temp->data);
    }
    return root;
}

void inorderTraversal(struct Node *root)
{
    if (root != NULL)
    {
        inorderTraversal(root->left);
        printf("%d ", root->data);
        inorderTraversal(root->right);
    }
}

int main()

```

```

{
    struct Node *root = NULL;
    int elements[] = {18, 15, 40, 50, 30, 17, 41};
    int numElements = sizeof(elements) / sizeof(elements[0]);
    int i;
    int elementsToDelete[] = {15, 17, 41};
    int numElementsToDelete = sizeof(elementsToDelete) / sizeof(elementsToDelete[0]);
    clrscr();

    for (i = 0; i < numElements; i++)
    {
        root = insert(root, elements[i]);
        printf("Tree after inserting %d: ", elements[i]);
        inorderTraversal(root);
        printf("\n");
    }
    root = insert(root, 45);
    printf("Tree after inserting 45: ");
    inorderTraversal(root);
    printf("\n");
    root = insert(root, 19);
    printf("Tree after inserting 19: ");
    inorderTraversal(root);
    printf("\n");

    for (i = 0; i < numElementsToDelete; i++)
    {
        root = deleteNode(root, elementsToDelete[i]);
        printf("Tree after deleting %d: ", elementsToDelete[i]);
        inorderTraversal(root);
        printf("\n");
    }
    getch();
    return 0;
}

```

OUTPUT:


```

Tree after inserting 18: 18
Tree after inserting 15: 15 18
Tree after inserting 40: 15 18 40
Tree after inserting 50: 15 18 40 50
Tree after inserting 30: 15 18 30 40 50
Tree after inserting 17: 15 17 18 30 40 50
Tree after inserting 41: 15 17 18 30 40 41 50
Tree after inserting 45: 15 17 18 30 40 41 45 50
Tree after inserting 19: 15 17 18 19 30 40 41 45 50
Tree after deleting 15: 17 18 19 30 40 41 45 50
Tree after deleting 17: 18 19 30 40 41 45 50
Tree after deleting 41: 18 19 30 40 45 50

```

LAB PROGRAM: 15

/*Write a program to create binary search tree with the elements {2,5,1,3,9,0,6} and perform inorder,preorder and postorder traversal*/

```

#include<stdio.h>
#include<conio.h>
struct treenode
{
    int data;
    struct treenode*left;
    struct treenode*right;
};
struct treenode*createnode(int data)
{
    struct treenode*newnode=(struct treenode*)malloc(sizeof(struct treenode));
    newnode->data=data;
    newnode->left=newnode->right=NULL;
    return newnode;
}
struct treenode*insert(struct treenode*root,int data)
{
    if(root==NULL)
    {
        return createnode(data);
    }
    if(data<root->data)
    {
        root->left=insert(root->left,data);
    }
    else if (data<root->data)
    {
        root->left=insert(root->left,data);
    }
    else if(data>root->data)
    {

```

```

root->right=insert(root->right,data);
}
return root;
}
void inorder(struct treenode*root)
{
if(root!=NULL)
{
inorder(root->left);
printf("%d",root->data);
inorder(root->right);
}
}
void preorder(struct treenode*root)
{
if(root!=NULL)
{
printf("%d",root->data);
preorder(root->left);
preorder(root->right);
}
}
void postorder(struct treenode*root)
{
if(root!=NULL)
{
postorder(root->left);
postorder(root->right);
printf("%d",root->data);
}
}
void main()
{
struct treenode*root=NULL;
int element[]={2,5,1,3,9,0,6},i;
int numelement=sizeof(element)/sizeof(element[0]);
clrscr();
for(i=0;i<numelement;i++)
{
root=insert(root,element[i]);
}
printf("Inorder traversal:");
inorder(root);
printf("\n");

```

```

printf("Preorder traversal:");
preorder(root);
printf("\n");
printf("Postorder traversal:");
postorder(root);
printf("\n");
getch();
}

```

OUTPUT:

```

C:\TURBOC3\BIN\PROJECT\b
In-order traversal: 0 1 2 3 5 6 9
Pre-order traversal: 2 1 0 5 3 9 6
Post-order traversal: 0 1 3 6 9 5 2

-----
Process exited after 0.02877 seconds with return value 0
Press any key to continue . . .

```

LAB PROGRAM: 16

/*Write a program to Sort the following elements using heap sort {9,16,32,8,4,1,5,8,0}*/

```

#include<stdio.h>
#include<conio.h>
void heapify(int arr[],int n,int i)
{
    int largest=i;
    int leftchild=2*i+1;
    int rightchild=2*i+2;
    if(leftchild<n&&arr[leftchild]>arr[largest])
    {
        largest=leftchild;
    }
    if(rightchild<n&&arr[rightchild]>arr[largest])
    {
        largest=rightchild;
    }
    if(largest!=i)
    {
        int temp=arr[i];
        arr[i]=arr[largest];
        arr[largest]=temp;
        heapify(arr,n,largest);
    }
}

```

```

}
void heapsort(int arr[],int n)
{
int i;
for(i=n/2-1;i>=0;i--)
{
heapify(arr,n,i);
}
for(i=n-1;i>0;i--)
{
int temp=arr[0];
arr[0]=arr[i];
arr[i]=temp;
heapify(arr,i,0);
}
}
void print(int arr[],int size)
{
int i;
for(i=0;i<size;i++)
{
printf("%d\t",arr[i]);
}
printf("\n");
}
void main()
{
int arr[]={9,16,32,8,4,1,5,8,0};
int n=sizeof(arr)/sizeof(arr[0]);
clrscr();
printf("Original array:\n");
print(arr,n);
heapsort(arr,n);
printf("Sorted array:\n");
print(arr,n);
getch();
}

```

OUTPUT:

```

Original array: 9 16 32 8 4 1 5 8 0
Sorted array: 0 1 4 5 8 8 9 16 32

```

LAB PROGRAM: 17

/*Given S1={"Flowers"} ; S2={"are beautiful"} I. Find the length of S1 II. Concatenate S1 and S2 III. Extract the substring "low" from S1 IV. Find "are" in S2 and replace it with "is"*/

```

#include<stdio.h>
#include<string.h>
#include<conio.h>
int main()
{
    char s1[]="Flowers";
    char s2[]="are beautiful";
    int lengths1=strlen(s1);
    char substring[10];
    char concatenated[50];
    char*found=strstr(s2,"are");
    clrscr();
    printf("1. Length of s1:%d\n",lengths1);
    strcpy(concatenated,s1);
    strcat(concatenated," ");
    strcat(concatenated,s2);
    printf("2. Concatenated string :%s\n",concatenated);
    strncpy(substring,s1+1,3);
    substring[3]='\0';
    printf("3. Extracted substring from s1:%s\n",substring);
    if(found!=NULL)
    {
        strncpy(found,"is",2);
        found[2]='\0';
    }
    printf("4. Modified s2:%s",s2);
    getch();
    return 0;
}

```

OUTPUT:

```

I. Length of S1: 7
II. Concatenated string: Flowers are beautiful
III. Extracted substring from S1: wer
IV. Modified S2: is

```